

이종 게임 MOD 패키지의 다중 출처 계보 복원을 위한 서버 기반 Provenance 분석 방법

Abstract. 게임 MOD는 Java bytecode와 같은 실행 코드뿐 아니라 JSON 등의 구조화 리소스, 이미지 및 설정 파일을 함께 포함하는 이종 소프트웨어 패키지이다. 실제 MOD 재사용 과정에서는 하나의 MOD 전체가 그대로 복제되기보다 여러 기존 MOD의 일부 구성요소가 조합되거나 수정되어 새로운 패키지가 형성될 수 있다. 이러한 경우 기존의 파일 단위 clone detection 또는 일대일 유사도 분석만으로는 각 구성요소의 출처와 패키지 전체의 다중 출처 관계를 복원하기 어렵다.

본 연구에서는 서버에 등록된 참조 MOD 집합을 대상으로 업로드된 MOD 패키지의 각 구성요소를 기존 부모 MOD 또는 등록되지 않은 UNKNOWN 출처로 귀속하고, 패키지 전체의 다중 부모 provenance를 복원하는 Hierarchical Multi-Parent Provenance Reconstruction 방법을 제안한다. 제안 방법은 CODE, STRUCTURED, IMAGE의 서로 다른 modality에서 identity-neutral 콘텐츠 증거를 추출한 후 전역 부모 후보를 검색하고, 패키지 수준에서 부모 집합과 부모 수를 먼저 추론한 뒤 구성요소별 출처를 할당한다. Java class dependency topology는 콘텐츠 기반 후보 집합 내부에서 구조적 일관성을 보완하는 약한 refinement로 사용한다.

실제 공개 게임 MOD 프로젝트를 기반으로 120개 프로젝트의 corpus를 구축하고, 완전히 분리된 360개 TEST 질의 패키지를 생성하였다. 각 질의는 5개의 CODE, 1개의 STRUCTURED, 1개의 IMAGE 구성요소로 구성되며 단일 부모, 다중 부모, known/unknown 혼합 상황을 포함한다. 최종 TEST에서 제안 방법은 Component Accuracy 0.8060, Parent F1 0.8442, UNKNOWN F1 0.7538을 기록하였다. 독립적인 구성요소 판정과 비교하면 계층적 콘텐츠 복원은 Exact Parent-Set Match를 0.2861에서 0.4139로, K Accuracy를 0.3250에서 0.4806으로 향상시켰다. 반면 dependency topology의 추가 효과는 제한적이었으며 Parent F1 차이에 대한 bootstrap 95% 신뢰구간은 0을 포함하였다.

FastAPI 기반 서버를 구현하여 offline 분석과 server-side 분석의 일치성을 확인하였으며, materialized package를 입력으로 한 end-to-end 중앙 처리 시간은 약 26.88 ms였다. 본 연구는 저작권 침해 여부를 자동 판정하는 것이 아니라 등록된 참조 콘텐츠와 업로드된 MOD 사이의 기술적 provenance 관계를 복원하여 재사용 출처 확인 및 후속 저작권 검토를 지원하는 것을 목적으로 한다.

Keywords: Game MOD · Software Provenance · Multi-Parent Reconstruction · Software Reuse · Copyright Forensics · Game Server

1 Introduction

게임 MOD 생태계에서는 기존 프로젝트의 코드, 설정 및 시각 리소스가 다른 MOD에 재사용될 수 있다. 이 재사용은 하나의 프로젝트 전체가 복제되는 형태뿐 아니라 여러 출처의 일부 구성요소가 새로운 패키지 하나에 혼합되는 형태로도 발생할 수

있다. 따라서 “질의 MOD가 어떤 하나의 MOD와 유사한가”를 판정하는 것만으로는 실제 재사용 구조를 충분히 설명하기 어렵다.

기존 clone detection 및 software reuse 연구는 pairwise similarity와 clone pair 검출에 강점을 갖는다 [?, ?, ?]. 그러나 게임 MOD는 CODE뿐 아니라 STRUCTURED 및 IMAGE 리소스를 함께 포함하고, 질의 하나가 여러 부모와 등록되지 않은 UNKNOWN을 동시에 포함할 수 있다. 본 연구는 이 문제를 component-level attribution과 package-level parent-set reconstruction을 동시에 수행하는 open-set multi-parent provenance 문제로 정의한다.

본 연구의 핵심 기여는 다음과 같다.

1. 게임 MOD를 CODE, STRUCTURED, IMAGE로 구성된 heterogeneous package로 모델링한다.
2. global parent candidate retrieval과 package-level parent-set inference를 결합한 hierarchical multi-parent reconstruction을 제안한다.
3. 실제 공개 MOD의 current/historical release에서 자동으로 component-level ground truth와 multi-parent/open-set benchmark를 생성한다.
4. FastAPI 기반 server-side pipeline을 구현하고 correctness, latency, throughput 및 gallery scalability를 평가한다.

2 Related Work

2.1 Software Clone Detection

NiCad는 normalization과 pretty-printing을 이용한 source-level near-miss clone detector이며 [?], SourcererCC는 대규모 source repository에서 token 기반 clone detection을 수행한다 [?]. StoneDetector는 Java source와 bytecode를 지원하는 clone detector로 본 연구의 CODE_BINARY external baseline과 직접 관련된다 [?].

2.2 Software Reuse and Origin Analysis

MOVERY는 수정된 OSS component의 vulnerable clone을 탐지하고 [?], LibScan은 Android application의 third-party library를 식별하며 [?], VOFinder는 공개 software vulnerability의 origin을 추적한다 [?]. 이러한 연구는 component reuse와 origin tracing의 필요성을 보여주지만, 본 연구는 하나의 heterogeneous MOD package 안에서 복수의 등록 부모와 UNKNOWN을 동시에 복원한다는 점에서 차이가 있다.

2.3 Copyright-Oriented Provenance

본 연구에서 복원되는 parent는 기술적 provenance candidate이며 법적인 저작권자를 의미하지 않는다. 또한 provenance 탐지 자체가 저작권 침해를 의미하지 않는다. 본 시스템의 목적은 server-side technical evidence를 통해 재사용 출처 확인과 후속 copyright review를 지원하는 것이다.

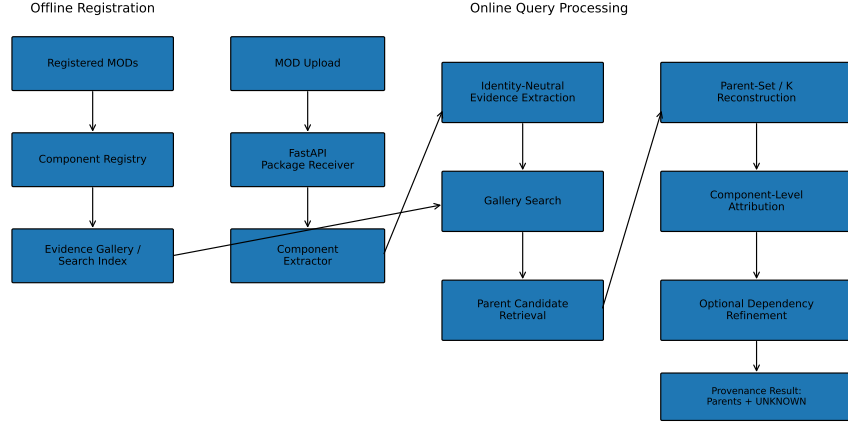


Fig. 1. 서버 기반 MOD provenance 분석의 전체 구조. 등록 gallery 구축과 online query processing을 분리하여 표현하였다.

3 Problem Definition and System Model

3.1 Multi-Parent Open-Set Problem

등록된 참조 MOD 집합을

$$\mathcal{G} = \{G_1, G_2, \dots, G_N\}$$

이라 하고 질의 MOD G_q 의 component set을

$$V_q = \{v_1, v_2, \dots, v_m\}$$

이라 한다. 각 component의 label은

$$y_i \in \{1, \dots, N, \text{UNKNOWN}\}$$

이며, 하나의 query는 둘 이상의 부모를 동시에 가질 수 있다. 질의의 실제 부모 수를 K 로 정의한다.

3.2 Server-Side Usage Scenario

서버는 등록 MOD의 component registry와 evidence gallery를 사전에 구축한다. 새 MOD가 업로드되면 package receiver가 파일을 수신하고 component extractor가 CODE, STRUCTURED, IMAGE를 분리한다. 이후 identity-neutral evidence를 생성하여 gallery search를 수행하고, global parent candidate와 parent set을 복원한 뒤 component-level label과 UNKNOWN을 반환한다.

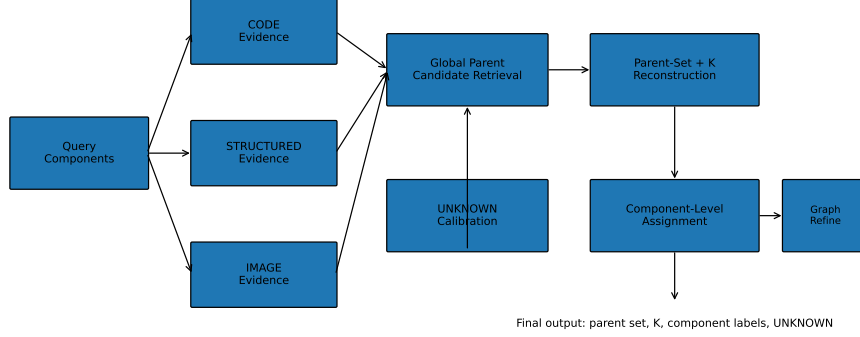


Fig. 2. Hierarchical Multi-Parent Provenance Reconstruction의 처리 흐름.

4 Proposed Method

4.1 Heterogeneous Content Evidence

CODE는 opcode 및 구조적 통계를 사용하며 path, class name, method descriptor, 문자열 및 constant-pool operand와 같이 identity leakage가 큰 정보를 제외한다. STRUCTURED는 JSON 등의 구조적 특성을 사용하고, IMAGE는 파일명 대신 decode된 pixel content를 사용한다.

4.2 Hierarchical Parent Reconstruction

개별 component를 독립적으로 판정하면 소수의 오탐이 package parent set에 불필요한 부모를 추가할 수 있다. 이를 완화하기 위해 먼저 전체 component evidence를 이용하여 global candidate를 검색하고, 후보 집합 안에서 parent set과 K 를 결정한 후 component attribution을 수행한다.

$$\hat{y} = \arg \max_y \left[\sum_{v \in V_q} \phi_v(y_v) + \lambda \sum_{(u,v) \in E_q} \psi_{uv}(y_u, y_v) - \gamma \Omega(y) \right].$$

여기서 ϕ_v 는 content evidence, ψ_{uv} 는 dependency consistency, $\Omega(y)$ 는 parent proliferation penalty이다.

4.3 UNKNOWN and Dependency Refinement

UNKNOWN threshold는 calibration split에서 modality별로 결정하고 TEST 이전에 freeze한다. Dependency topology는 주요 identity evidence가 아니라 content candidate pool 내부에서만 약한 refinement로 사용한다. 초기 connected-component 강제 방식은 cross-parent bridge stress에서 취약했기 때문에 최종 방법에서는 strong grouping을 사용하지 않는다.

Table 1. 프로젝트 단위 최종 분할

Split	Projects
CALIBRATION_KNOWN	25
TEST_KNOWN	45
UNKNOWN_HELDOUT	20
CALIBRATION_BACKGROUND	15
TEST_BACKGROUND	15

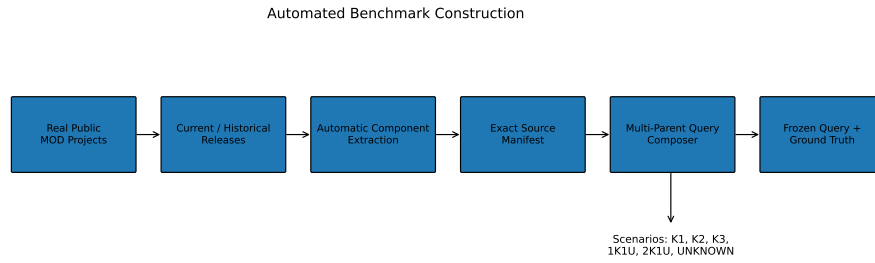


Fig. 3. 실제 공개 MOD에서 frozen multi-parent/open-set benchmark를 생성하는 자동화 파이프라인.

5 Benchmark and Experimental Design

5.1 Real MOD Corpus and Split

실험 corpus는 실제 공개 MOD 프로젝트 120개로 구성된다. 이 중 90개는 target, 30개는 background 역할을 갖는다.

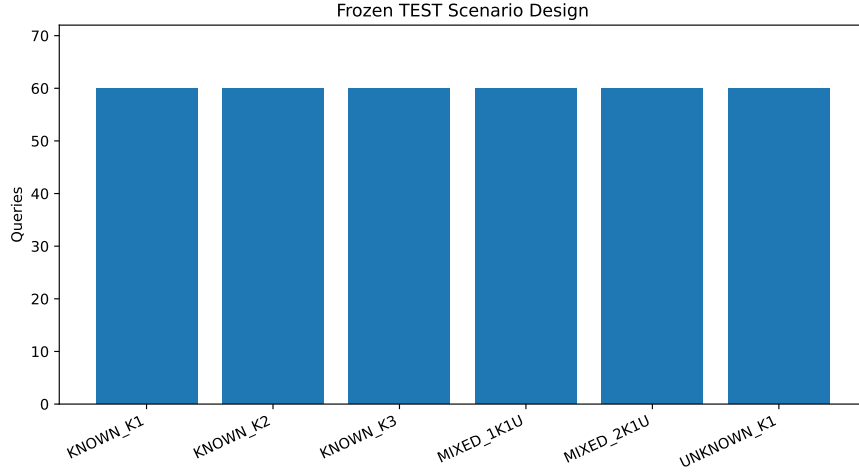
CALIBRATION_KNOWN과 CALIBRATION_BACKGROUND는 threshold 및 reconstruction parameter 결정에만 사용한다. TEST_KNOWN과 TEST_BACKGROUND는 최종 평가에 사용하고, UNKNOWN_HELDOUT은 gallery에 존재하지 않는 open-set source로 사용한다. TEST 결과를 본 뒤 parameter를 재조정하지 않는다.

5.2 Automated Benchmark Construction

current/historical release에서 component를 자동 추출하고 exact source manifest를 기록한다. 이후 여러 source project에서 component를 선택하여 query를 구성하고 payload hash와 source identity를 private manifest로 고정한다. 이 방식은 수작업 labeling 없이 component-level ground truth를 재현할 수 있도록 한다.

Table 2. Frozen TEST scenario 구성

Scenario	Queries	Known Parents	Unknown Parents
KNOWN_K1	60	1	0
KNOWN_K2	60	2	0
KNOWN_K3	60	3	0
MIXED_1K1U	60	1	1
MIXED_2K1U	60	2	1
UNKNOWN_K1	60	0	1

**Fig. 4.** 최종 TEST의 6개 scenario 구성. 각 scenario는 60개의 query로 고정하였다.

5.3 Query Composition and Scenarios

전체 benchmark는 calibration query 180개와 TEST query 360개, 총 540개이다. 각 query는 정확히 7개의 component를 가지며 CODE_BINARY 5개, STRUCTURED 1개, IMAGE 1개로 구성된다.

최종 TEST에는 다음 여섯 시나리오가 각각 60개씩 포함된다.

TEST의 K 분포는 $K = 1, 2, 3$ 이 각각 120개로 균형을 이룬다. 현재 primary benchmark는 query당 최대 하나의 held-out UNKNOWN source를 사용하며 multi-UNKNOWN clustering은 후속 보강 실험으로 둔다.

5.4 Frozen Calibration Protocol

모든 threshold와 hyperparameter는 calibration에서 결정한 뒤 TEST 이전에 고정하였다. 최종 주요 설정은 CODE threshold 0.1302083, STRUCTURED threshold 0.03125, IMAGE threshold 0, $\alpha = 0.75$, regret weight 0.25, $\lambda = 0.5$, candidate limit $M = 10$, $\beta = 0.1$, Top- $R = 3$, $K_{\max} = 3$ 이다.

Table 3. 최종 TEST 성능

Metric	Score
Component Accuracy	0.8060
UNKNOWN F1	0.7538
Parent F1	0.8442
Exact Parent-Set Match	0.4194
K Accuracy	0.4861

Table 4. Ablation 결과

Method	Comp. Acc.	Parent F1	Exact K	Acc.
Independent	0.7758	0.7974	0.2861	0.3250
Hierarchical Content	0.8071	0.8435	0.4139	0.4806
Final + Graph	0.8060	0.8442	0.4194	0.4861

5.5 Baselines and Metrics

내부 baseline은 (B0) Independent Attribution, (B1) Hierarchical Content, (B2) Final + Graph로 구성한다. B0와 B1의 차이는 package-level hierarchy의 기여를, B1과 B2의 차이는 dependency refinement의 incremental effect를 측정한다.

Component 수준에서는 Component Accuracy와 UNKNOWN F1을 사용한다. Package 수준에서는 Parent F1, Exact Parent-Set Match, K Accuracy를 사용한다. 통계적 불확실성은 10,000회 bootstrap과 source-project cluster bootstrap으로 평가한다.

6 Experimental Results

6.1 Overall and Ablation Results

Hierarchical Content는 Independent 대비 Exact Parent-Set Match를 0.2861에서 0.4139로, K Accuracy를 0.3250에서 0.4806으로 향상시켰다. 반면 graph 추가 효과는 작았다.

6.2 Dependency Refinement and Statistical Significance

Content-only Parent F1은 0.843545, 최종 graph 모델은 0.844233으로 차이는 약 0.000688이다. 10,000회 bootstrap에서 이 차이의 95% CI는 $[-0.001693, 0.003466]$ 으로 0을 포함한다. 따라서 graph가 TEST 성능을 유의하게 향상시켰다고 주장하지 않는다.

최종 모델의 bootstrap 95% CI는 Component Accuracy $[0.787698, 0.823810]$, Parent F1 $[0.829101, 0.859114]$, Exact Parent-Set Match $[0.372222, 0.466667]$, K Accuracy $[0.436111, 0.533333]$, UNKNOWN F1 $[0.732551, 0.774445]$ 이다.

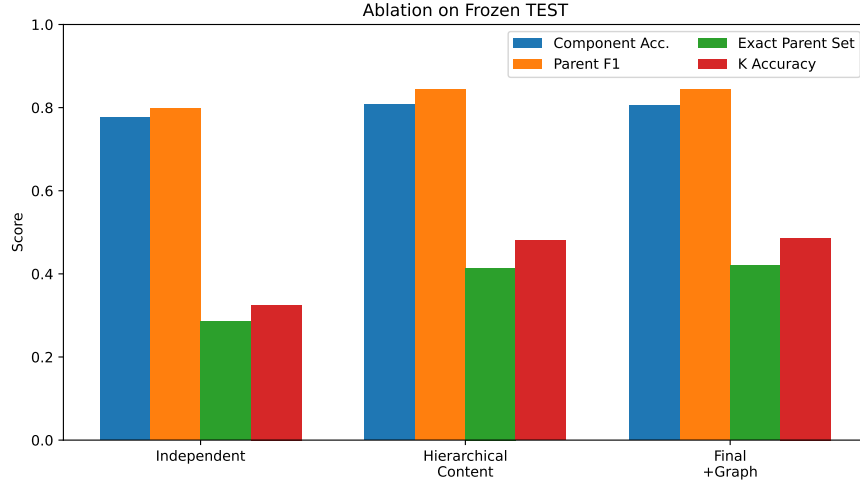


Fig. 5. Independent attribution, hierarchical content reconstruction, graph refinement의 성능 비교.

6.3 Candidate Retrieval and Oracle K

Candidate mean recall은 0.974444이며 실제 부모가 candidate pool에 모두 포함된 query 비율은 0.943333이다. Oracle K에서 Component Accuracy 0.825794, Parent F1 0.872976, Exact Parent-Set Match 0.558333, K Accuracy 0.627778, UNKNOWN F1 0.790948을 얻었다. 이는 현재 주요 병목이 candidate retrieval 보다는 정확한 parent-count inference에 있음을 시사한다.

6.4 External Baseline Status

NiCad는 Java source가 실제로 매핑 가능한 subset에 한하여 auxiliary sensitivity baseline으로 사용한다. TEST CODE 1,800개 중 source-resolvable component는 1,169개였다.

StoneDetector는 CODE_BINARY external baseline으로 사용한다. frozen external corpus는 query CODE 1,800개와 60개 gallery project의 10,448개 CODE class, 총 12,248개 class이다. 공식 구현은 최신 Java class version에서 기존 Soot/ASM compatibility 문제가 확인되었고, detector threshold 및 metric을 유지한 dependency-modernized reproduction을 준비하였다. Java 21/25 probe는 통과했으며, 현재 THREADSIZE=1 deterministic pilot과 60-parent 전체 실행이 남아 있다. 외부 baseline 결과는 TEST tuning 없이 추가한다.

7 Server-Side Evaluation

7.1 Server Architecture and Correctness

Fig. ??의 FastAPI pipeline을 실제 TEST query에 적용하였다. Offline reconstruction과 server-side reconstruction을 비교한 결과 360/360 query가 동일하였다.

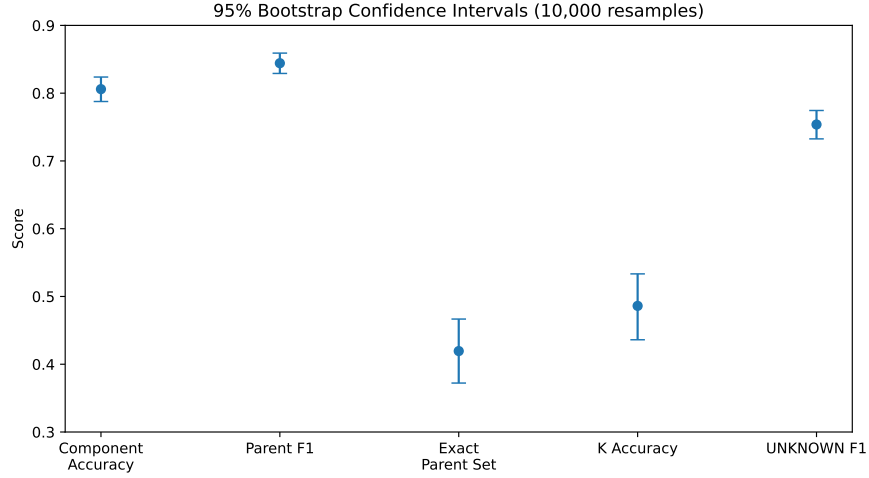


Fig. 6. 최종 TEST metric의 10,000회 bootstrap 95% confidence interval.

7.2 End-to-End Latency

Materialized package에서 extraction, search, reconstruction을 모두 수행하는 조건에서 total p50은 26.88 ms였다. Extraction은 14.44 ms, search는 10.69 ms, reconstruction은 1.086 ms였다.

7.3 Concurrency and Throughput

Precomputed condition에서 최대 약 86.21 req/s를 관찰하였다. Evidence-to-search-to-reconstruct 조건에서는 concurrency 4에서 약 127.92 req/s를 기록하였다. Materialized package end-to-end 조건에서는 concurrency 16에서 약 38.71 req/s를 기록했고 실패 요청은 0건이었다.

7.4 Gallery Scalability

Gallery 규모를 20개에서 100개 project로 증가시키면 search p50은 4.35 ms에서 21.46 ms로 증가하였다. 현재 reconstruction 비용보다 gallery search 증가가 더 큰 scalability bottleneck으로 나타났다.

8 Discussion

가장 중요한 결과는 component-level similarity 자체보다 package-level hierarchical reconstruction이 parent-set 복원에 실질적으로 기여했다는 점이다. Graph topology의 incremental effect는 통계적으로 뚜렷하지 않았으므로 최종 논문의 중심 기여는 heterogeneous content evidence와 hierarchical parent reconstruction에 둔다.

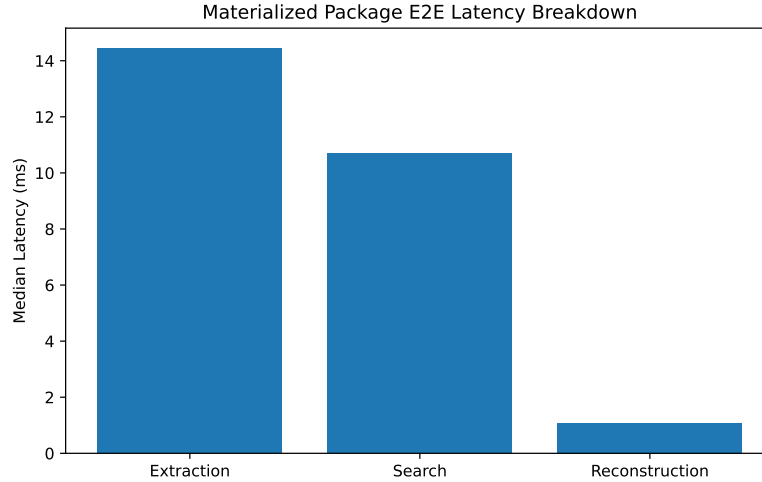


Fig. 7. Materialized MOD package의 end-to-end latency breakdown.

Candidate retrieval recall이 높은 반면 Oracle K에서 성능이 크게 상승한 점은 parent-count inference가 다음 핵심 연구 문제임을 보여준다. 서버 관점에서는 ANN 또는 다른 index를 통한 gallery search 개선이 가장 직접적인 확장 방향이다.

본 연구의 결과는 copyright infringement 판정이 아니라 technical provenance evidence이다. 법적 소유권 및 이용허락 여부는 후속 human review가 필요하다.

9 Research Status and Future Plan

9.1 Completed

현재 corpus 구축, historical/current component registry, frozen split, heterogeneous evidence, multi-parent/open-set benchmark, calibration, 최종 TEST 360 개, ablation, bootstrap, source-project cluster bootstrap, FastAPI server correctness/latency/throughput/scalability 평가까지 완료하였다.

9.2 In Progress

현재 가장 중요한 진행 중 작업은 StoneDetector CODE_BINARY external baseline이다. 공식 implementation의 최신 bytecode compatibility 문제를 확인했고 dependency-modernized reproduction과 Java 21/25 probe까지 완료하였다. 다음 단계는 THREADSIZE=1 pilot 검증과 60-parent full run이다.

9.3 Required Next Work

논문 제출 전 필수적으로 StoneDetector full baseline, NiCad source-resolvable subset baseline, 외부 baseline 결과의 통계 비교, 관련 연구 서지정보 최종 검증, figure/table 정리 및 영문 LNCS 원고 변환을 수행한다.

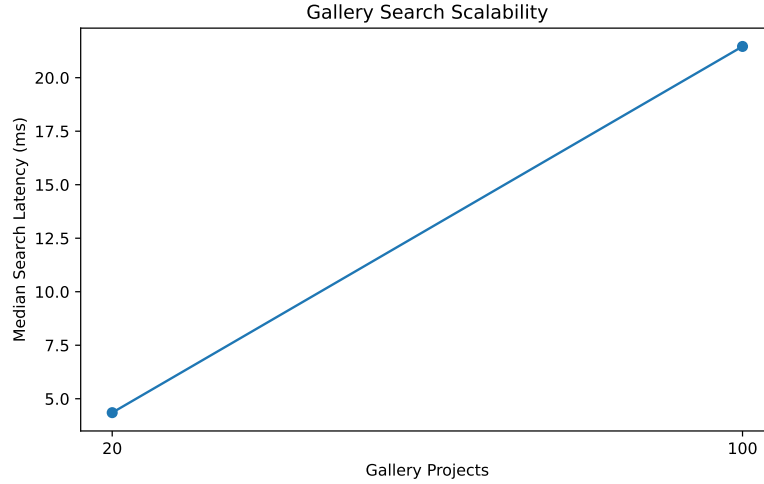


Fig. 8. Gallery project 수 증가에 따른 search median latency 변화.

9.4 Optional Strengthening

시간이 허용되면 multi-UNKNOWN, 새로운 holdout을 이용한 K inference 개선, ANN gallery search, 다른 MOD ecosystem 확장을 수행한다.

9.5 Advisor Confirmation Points

현재 지도교수 확인이 필요한 사항은 세 가지이다. 첫째, 논문의 중심 기여를 dependency graph 자체보다 heterogeneous content evidence와 hierarchical multi-parent reconstruction으로 설정하는 방향이다. 둘째, copyright framing을 infringement decision이 아닌 technical provenance 및 review support로 설정하는 방향이다. 셋째, 현재 frozen TEST/server evaluation에 StoneDetector와 NiCad 외부 baseline을 추가하는 수준을 본 논문의 필수 실험 범위로 확정할지 여부이다.

10 Conclusion

본 연구는 heterogeneous game MOD package에서 component-level source와 package-level multi-parent provenance를 복원하는 server-side 방법을 제안한다. 최종 TEST에서 Component Accuracy 0.8060, Parent F1 0.8442, UNKNOWN F1 0.7538을 기록했으며, hierarchical reconstruction은 independent baseline보다 Exact Parent-Set Match와 K Accuracy를 크게 개선하였다. Dependency graph의 추가 효과는 제한적이었다고, 현재 추론 병목은 parent-count inference, 시스템 확장성 병목은 gallery search로 나타났다.

향후 외부 baseline을 완료한 뒤 multi-UNKNOWN, improved K inference, ANN-based search 및 다른 MOD 생태계로 평가를 확장한다.

References

1. Roy, C.K., Cordy, J.R.: NiCad: Accurate Detection of Near-Miss Intentional Clones Using Flexible Pretty-Printing and Code Normalization. ICPC (2008)
2. Sajnani, H., Saini, V., Svajlenko, J., Roy, C.K., Lopes, C.V.: SourcererCC: Scaling Code Clone Detection to Big Code. ICSE (2016)
3. Heinze, T.S., Schäfer, A., Amme, W.: StoneDetector: Conventional and Versatile Code Clone Detection for Java. arXiv:2508.03435 (2025)
4. Woo, S., Hong, H., Choi, E., Lee, H.: MOVERY: A Precise Approach for Modified Vulnerable Code Clone Discovery from Modified Open-Source Software Components. USENIX Security (2022)
5. Wu, Y., Sun, C., Zeng, D., Tan, G., Ma, S., Wang, P.: LibScan: Towards More Precise Third-Party Library Identification for Android Applications. USENIX Security (2023)
6. Woo, S., Lee, D., Park, S., Lee, H., Dietrich, S.: V0Finder: Discovering the Correct Origin of Publicly Reported Software Vulnerabilities. USENIX Security (2021)